

Signal Object Detection

Raport proiect Machine Learning

Nume: Pupaza Alexandra Maria

Grupa: 232

Modelul 1 – Regresie Logistica

Am inceput cu Regresie Logistica pentru a avea un punct simplu de comparatie. Pentru acest model am folosit imaginile convertite la grayscale, apoi le-am transformat in vectori. Am aplicat si `StandardScaler`, deoarece modelul lucreaza mai bine cu valori standardizate.

Rezultatul a fost slab, lucru destul de normal pentru acest tip de problema. Regresia Logistica este un model liniar, deci nu poate invata foarte bine formele si detaliile vizuale din imagini. Din acest motiv, am pastrat modelul doar ca baseline.

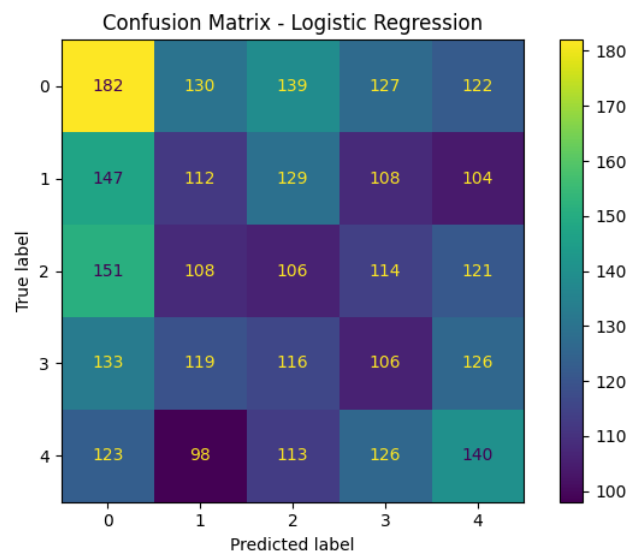


Figure 1: Matricea de confuzie pentru Regresie Logistica.

Modelul 2 – KNN + PCA

Pentru KNN am folosit tot imagini grayscale transformate in vectori. Pentru ca vectorii obtinuti din pixeli aveau dimensiune mare, am aplicat PCA cu 100 de componente. Astfel, modelul a putut calcula distantele mai usor.

Am folosit `n_neighbors = 7`, distanta euclidiană si `weights = "distance"`. Acuratetea obtinuta pe validation a fost de **21.51%**. Rezultatul este slab, deoarece KNN compara imaginile prin distante intre vectori si nu invata efectiv structura vizuala a imaginilor.

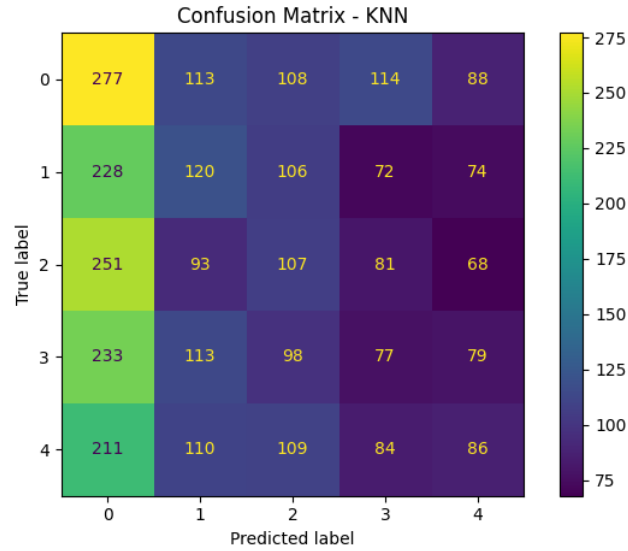


Figure 2: Matricea de confuzie pentru KNN.

Modelul 3 – Random Forest

Am testat si Random Forest, tot pe imagini grayscale transformate in vectori. Modelul a fost folosit ca o alta varianta clasica de comparatie.

Am incercat mai multe valori pentru `n_estimators` si `max_depth`. Cea mai buna configuratie a fost cu `n_estimators` = 300 si `max_depth` = 40, obtinand **27.06%** pe validation.

<code>n_estimators</code>	<code>max_depth</code>	Acuratete validation
100	20	26.13%
100	40	25.03%
100	None	25.48%
300	20	25.74%
300	40	27.06%
300	None	26.65%

Table 1: Rezultate pentru Random Forest.

Rezultatul este mai bun decat la KNN, dar inca slab pentru problema finala. Modelul nu foloseste suficient de bine pozitia si forma obiectelor din imagine.

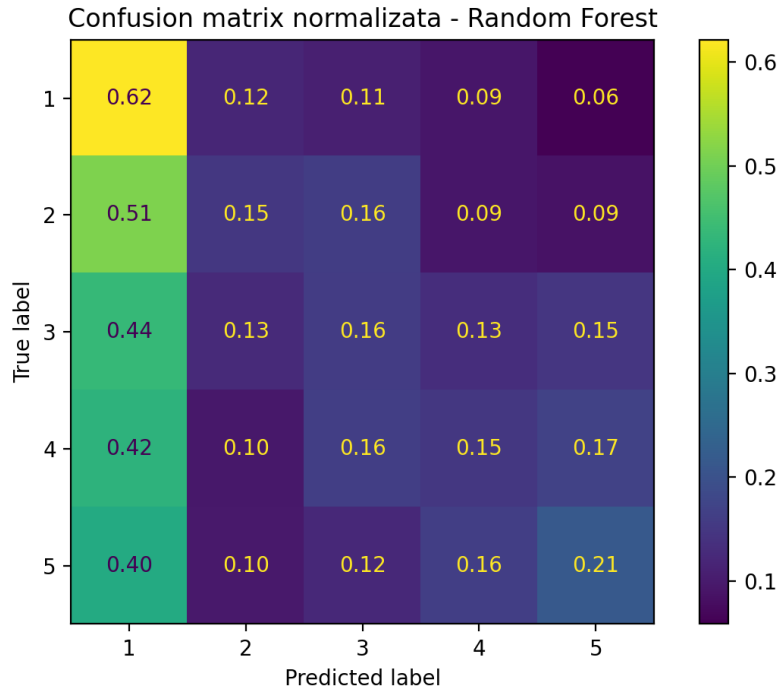


Figure 3: Matricea de confuzie pentru Random Forest.

Modelul 4 – Extra Trees pe features manuale

Dupa modelele pe pixeli raw, am incercat Extra Trees folosind features manuale. Pentru fiecare imagine am extras statistici simple, histograme de intensitate si proiectii pe randuri si coloane.

Aceasta abordare a mers mai bine, pentru ca modelul nu mai primea direct toti pixelii, ci o reprezentare mai compacta si mai usor de folosit.

n_estimators	max_depth	max_features	Acuratete validation
200	12	sqrt	47.71%
300	20	sqrt	47.77%
500	20	sqrt	48.00%
500	None	sqrt	47.71%
700	None	sqrt	47.61%

Table 2: Rezultate pentru Extra Trees.

Cea mai buna configuratie a fost `n_estimators = 500`, `max_depth = 20`, `max_features = "sqrt"`, cu acuratetea de **48.00%**. Se vede ca features-urile manuale au fost mult mai utile decat pixelii raw pentru acest tip de model.

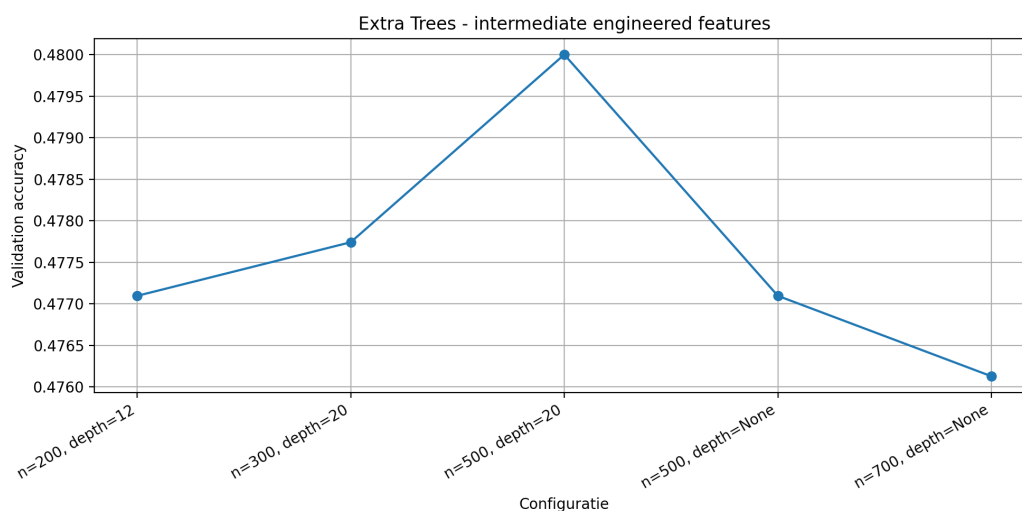


Figure 4: Comparatie intre configurările Extra Trees.

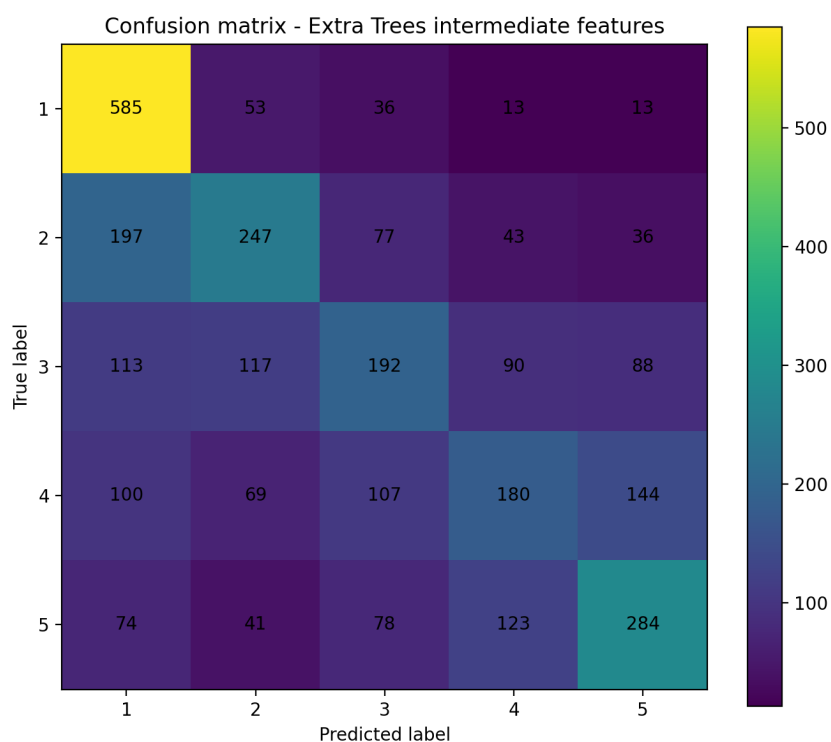


Figure 5: Matricea de confuzie pentru Extra Trees.

Modelul 5 – MLP pe features manuale

Am incercat apoi un MLP pe aceleasi tipuri de features manuale. Ideea a fost sa vad daca o retea fully-connected poate combina mai bine statisticile extrase din imagini.

Am folosit statistici pe imaginea grayscale, histograme, proiectii pe randuri si coloane, statistici RGB si diferente intre canalele de culoare. Dupa extragerea feature-urilor, am aplicat `StandardScaler` si am antrenat mai multe variante de MLP.

Rezultatele au fost mai bune decat la Extra Trees, deoarece MLP-ul poate combina neliniar features-urile. Totusi, modelul ramane limitat de faptul ca nu invata direct din imagine, ci doar din statistici extrase manual.

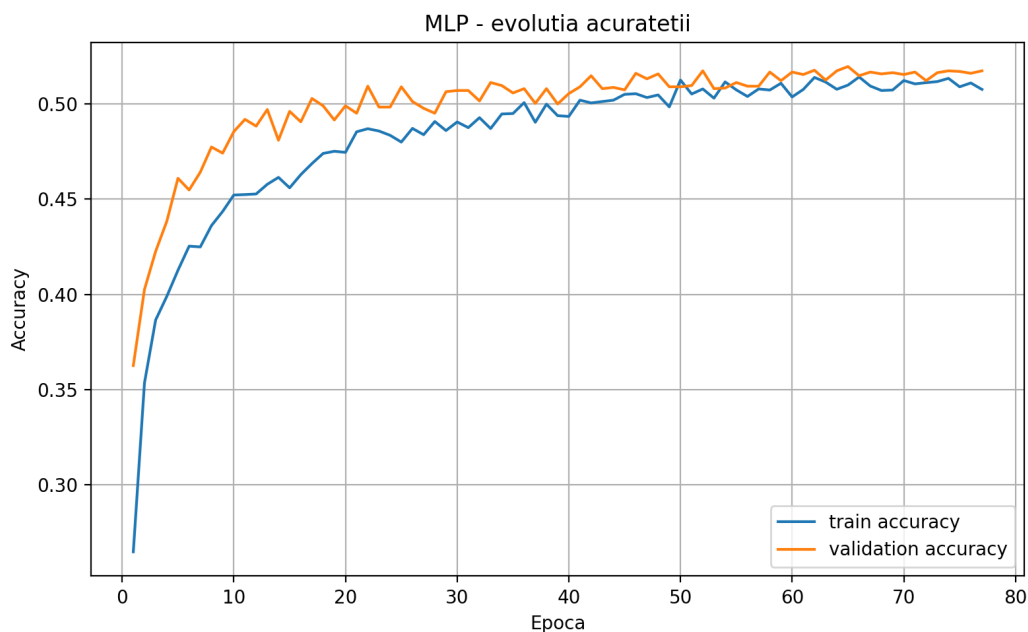


Figure 6: Evolutia acuratetii pentru MLP pe features manuale.

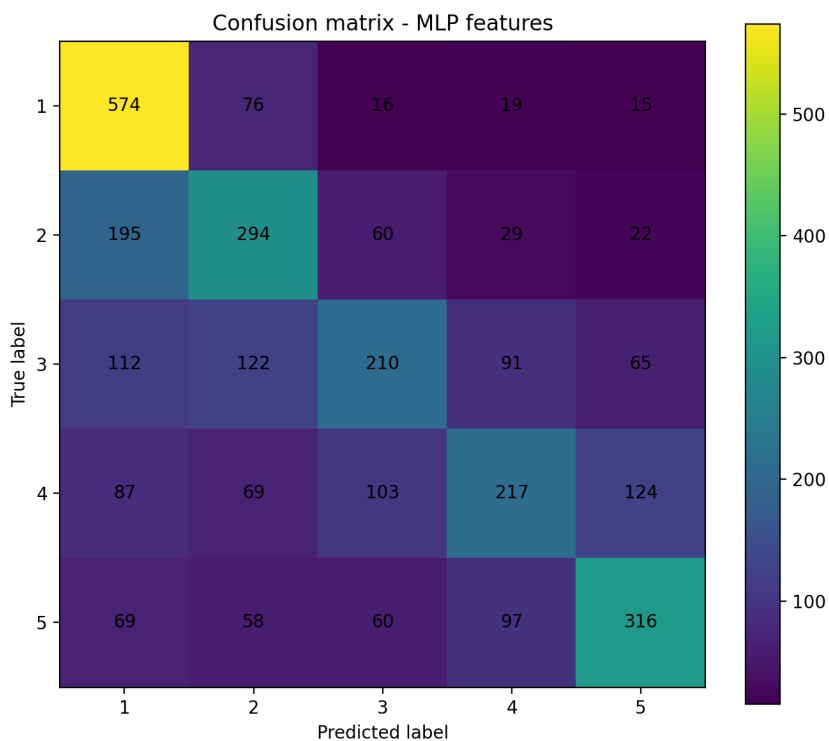


Figure 7: Matricea de confuzie pentru MLP pe features manuale.

Modelul 6 – Model hibrid CNN + features manuale

Am testat si un model hibrid, format din doua ramuri. Prima ramura procesa imaginea grayscale cu un CNN, iar a doua ramura procesa features-urile manuale cu un MLP. Cele doua rezultate erau concatenate si folosite pentru clasificarea finala.

Aceasta varianta a fost mai buna decat MLP-ul simplu, deoarece modelul putea folosi atat informatia directa din imagine, cat si statisticile extrase manual. Totusi, rezultatele cele mai bune au venit abia cand am trecut la modele CNN mai puternice pe imagini RGB.

Modelul 7 – Varianta 3: CNN custom + XGBoost pe embedding-uri

Prima varianta CNN + XGBoost a fost varianta 3. CNN-ul a fost folosit pentru a invata reprezentari vizuale din imaginile RGB, redimensionate la 128×128 .

Dupa antrenarea CNN-ului, am folosit TTA si am extras embedding-uri de dimensiune 512. Aceste embedding-uri au fost concatenate cu probabilitatile CNN si date mai departe unui model XGBoost. La final am testat si un fusion intre probabilitatile CNN si XGBoost.

Model	Validation accuracy
CNN fara TTA	0.64153
CNN cu TTA	0.64194
XGBoost pe embedding-uri CNN	0.65040
Fusion CNN + XGBoost	0.65403

Table 3: Rezultate pentru varianta 3.

In aceasta varianta, XGBoost a ajutat fata de CNN-ul simplu. Cea mai buna varianta a fost fusion-ul CNN + XGBoost, cu acuratetea **0.65403**.

Modelul 8 – Varianta 4: CNN imbunatatit + features manuale + XGBoost

In varianta 4 am pastrat ideea de CNN + XGBoost, dar am imbunatatit pipeline-ul. Imaginile au fost redimensionate la 160×72 , pentru a pastra mai bine forma initiala.

Fata de varianta 3, am schimbat arhitectura CNN si am adaugat un `count_head` auxiliar. Pentru XGBoost am folosit embedding-uri CNN, probabilitati CNN si features manuale extrase din imaginile grayscale. Vectorul final pentru XGBoost a avut dimensiunea 713.

Model	Validation accuracy
CNN fara TTA	0.70121
CNN cu TTA	0.70565
XGBoost	0.68831
Fusion CNN + XGBoost	0.70565

Table 4: Rezultate pentru varianta 4.

CNN-ul a crescut mult fata de varianta 3. Totusi, XGBoost nu a mai imbunatatit rezultatul. Cea mai buna pondere la fusion a fost $\alpha_{cnn} = 1.000$ si $\alpha_{xgb} = 0.000$, deci practic rezultatul final al variantei 4 a ramas CNN-ul singur.

Modelul 9 – Modelul final: CNN custom RGB + EMA + TTA

Pentru modelul final am continuat pe directia CNN, deoarece variantele anterioare au aratat ca imbunatatirea retelei convolutionale a avut cel mai mare impact.

Preprocesare

Pentru modelul final, imaginile au fost convertite in RGB si redimensionate la 192×84 . Pentru train am folosit augmentari simple: `RandomHorizontalFlip`, `RandomAffine`, `ColorJitter` si `RandomErasing`. In etapa de fine-tuning, augmentarile au fost mai usoare.

Arhitectura

Modelul primeste imagini RGB de forma:

$$(3, 192, 84)$$

Arhitectura contine:

- un bloc initial convolutional;
- mai multe blocuri reziduale;
- blocuri Squeeze-and-Excitation;
- DropPath si Dropout pentru regularizare;
- pooling global si straturi fully-connected;
- un head final pentru cele 5 clase.

Modelul foloseste si un `count_head` auxiliar in timpul antrenarii.

Loss si antrenare

Functia de pierdere combina clasificarea principala cu doua pierderi auxiliare:

$$loss = CE + 0.10 \cdot count_loss + 0.06 \cdot expected_class_loss$$

Am folosit **AdamW**, scheduler pentru learning rate si early stopping. Antrenarea s-a facut in doua etape: o etapa initiala si apoi fine-tuning cu learning rate mai mic. In timpul antrenarii am pastrat si o varianta EMA a modelului, pentru rezultate mai stabile.

Etapa	Epoci maxime	Learning rate	Weight decay	Label smoothing	Patience
Initial training	85	$3 \cdot 10^{-4}$	$4 \cdot 10^{-4}$	0.035	16
Fine-tuning	30	$2.5 \cdot 10^{-5}$	$2.5 \cdot 10^{-4}$	0.015	9

Table 5: Configuratia de antrenare pentru modelul final.

TTA

La final am testat si TTA. Pentru fiecare imagine s-au folosit patru variante: imaginea originala, imaginea intoarsa orizontal, imaginea marita usor si imaginea marita + intoarsa orizontal.

Predictia finala a fost calculata ca medie ponderata:

$$0.55 \cdot original + 0.25 \cdot flip + 0.12 \cdot scaled + 0.08 \cdot scaled_flip$$

Rezultate

Varianta model	Validation accuracy
CNN initial	0.72056
CNN initial EMA	0.72823
CNN dupa fine-tuning	0.73347
CNN EMA fara TTA	0.73226
CNN EMA cu TTA	0.73105

Table 6: Rezultatele modelului final pe validation.

Cel mai bun rezultat local a fost obtinut de CNN-ul dupa fine-tuning, cu acuratetea **0.73347**. Modelul EMA fara TTA a fost foarte apropiat, cu **0.73226**. TTA nu a imbunatatit rezultatul local, dar a fost pastrat ca experiment.

Din matricea de confuzie se observa ca modelul clasifica cel mai bine clasa 1. Cele mai multe confuzii apar intre clase apropiate vizual, mai ales intre clasele 4 si 5.

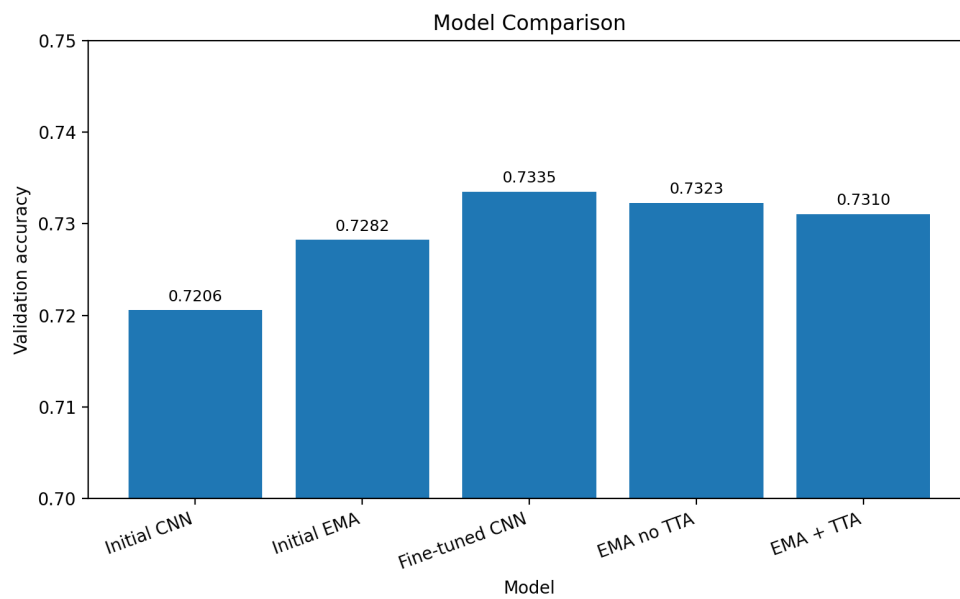


Figure 8: Comparatie intre variantele modelului final.

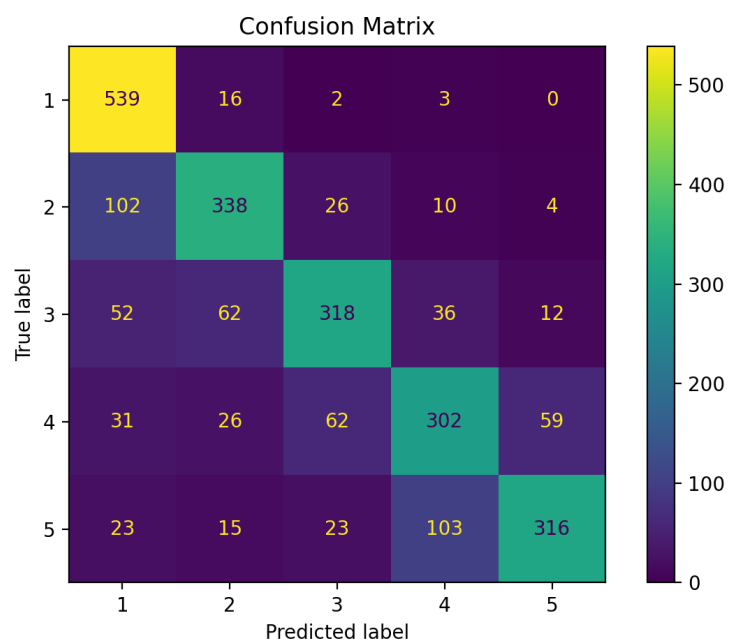


Figure 9: Matricea de confuzie pentru modelul final.

Compararea modelelor

Model	Rezultat local
Modelul 2 – KNN + PCA	21.51% validation
Modelul 3 – Random Forest	27.06% validation
Modelul 4 – Extra Trees	48.00% validation
Modelul 5 – MLP pe features manuale	aproximativ 0.52 validation
Modelul 7 – Varianta 3: CNN + XGBoost	0.65403 validation
Modelul 8 – Varianta 4: CNN imbunatatit	0.70565 validation
Modelul 9 – Model final	0.73347 validation

Table 7: Comparatie intre principalele modele testate.

Se observa ca modelele clasice au avut rezultate slabe pe pixeli raw. Rezultatele au crescut cand am folosit features manuale, dar cea mai mare imbunatatire a venit de la modelele CNN. De aceea, solutia finala aleasa a fost modelul CNN custom RGB cu EMA si fine-tuning.